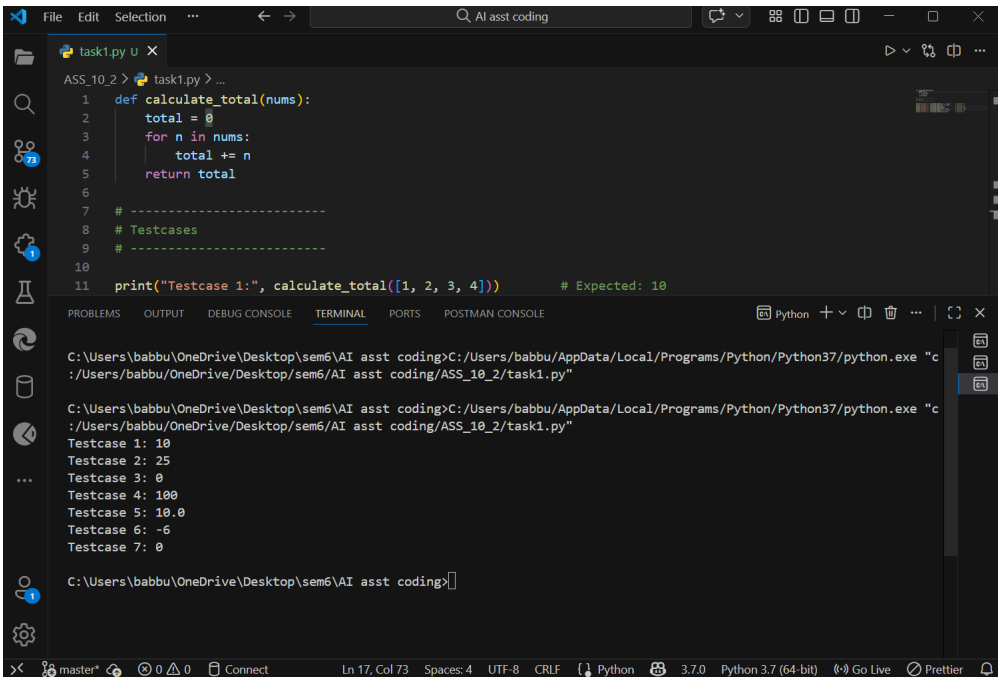
ASSIGNMENT 10.2
AI ASSISTED CODING

E.SAI ANURATH | BATCH-29 | 2303A51560 |
DATE:24-02-2026

> **Task Description -1(Error Detection and Correction)**
> Task:
> Use AI to analyze a Python script and correct all syntax and logical errors.
> **Sample Input Code:**
> def calculate_total(nums)
> sum = 0
> for n in nums
> sum += n
> return total
> **Expected Output-1:**
> Corrected and executable Python code with brief explanations of the identified syntax and logic errors.

```
def calculate_total(nums):
    total = 0
    for n in nums:
        total += n
    return total
```



REVIEW AND EXPLAINATION

1. Missing colon in function definition

❌ `def calculate_total(nums)`

✔️ `def calculate_total(nums):`

Why it's wrong: Python requires `:` after function headers.

---

2. Wrong variable name (`sum` used, but returned `total`)

❌ Code uses `sum = 0` but returns `total` → `total` was never defined

✔️ Renamed to a single consistent variable: `total`

Why it's wrong: Returning an undefined variable causes a `NameError`.

---

3. Missing colon in the for loop

❌ `for n in nums`

✔️ `for n in nums:`

Why it's wrong: Python requires `:` after control structures.

---

4. Logical Fix — `+=` should add to the correct variable

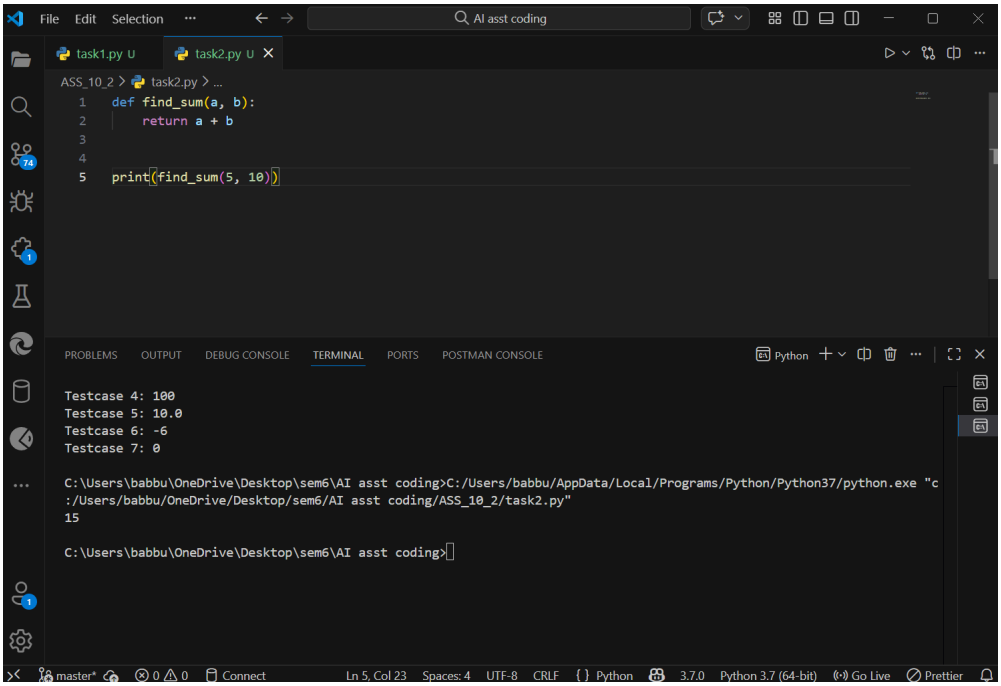
❌ If using `sum += n`, but returning `total`, it's inconsistent

✔️ Use **one correct accumulator**: `total += n`

> **Task Description -2(Code Style Standardization)**
> Task:
> Use AI to refactor Python code to comply with standard coding style guidelines.
> **Sample Input Code:**
> def findSum(a,b):return a+b
> print(findSum(5,10))
> **Expected Output-2:**
> Well-structured, consistently formatted Python code following standard style conventions.

```
def find_sum(a, b):
    return a + b


print(find_sum(5, 10))
```



REVIEW AND EXPLAINATION

In the given code, the function and the print statement were written on the same line, which makes the code hard to read. Also, the function name was in camelCase (`findSum`), but in Python we usually write function names in snake_case, so I changed it to `find_sum`. I also added proper spaces after the commas in the parameters.

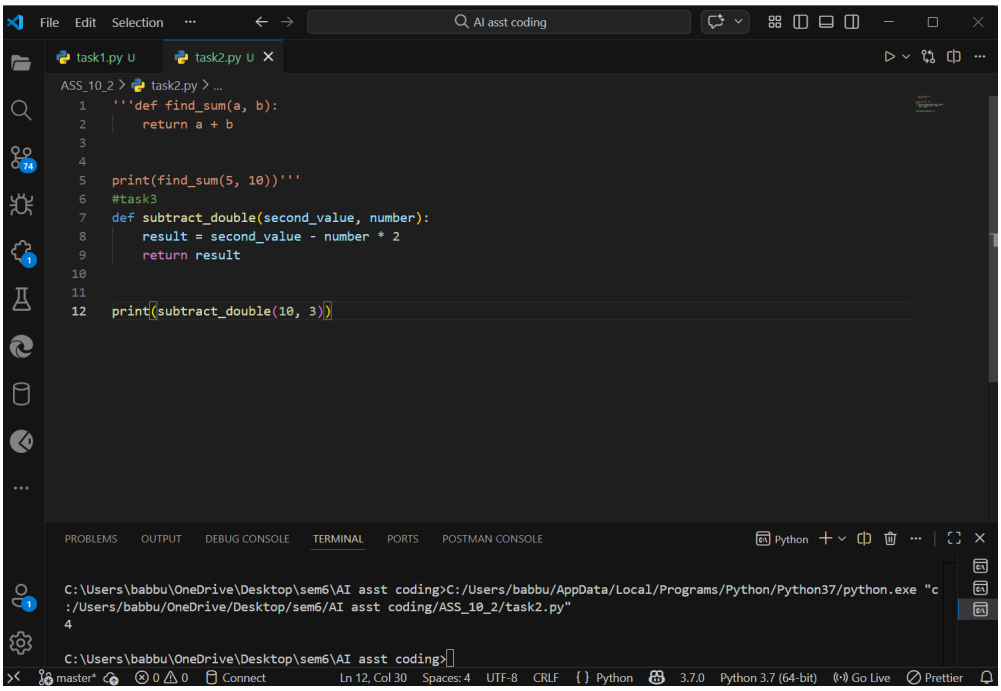
Next, I separated the function definition and the return statement onto separate lines to follow Python's standard formatting. Finally, I added a blank line before the print statement so the code looks cleaner and follows the usual PEP-8 style.

Overall, I just reorganized the code to make it readable, properly spaced, and consistent with standard Python style guidelines.

> **Task Description -3(Code Clarity Improvement)**
> Task:
> Use AI to improve code readability without changing its functionality.
> **Sample Input Code:**
> def f(x,y):
> return x-y*2
> print(f(10,3))
> **Expected Output-3:**
> Python code rewritten with meaningful function and variable names,
> proper indentation, and improved clarity.

```
def subtract_double(second_value, number):
    result = second_value - number * 2
    return result
```

```
print(subtract_double(10, 3))
```



REVIEW AND EXPLAINATION

In the original code, the function name `f` and the variable names `x` and `y` were not meaningful, so it was hard to understand what the code was doing. I changed the function name to `subtract_double` and used clearer variable names so that anyone reading the code can guess the purpose without thinking too much.

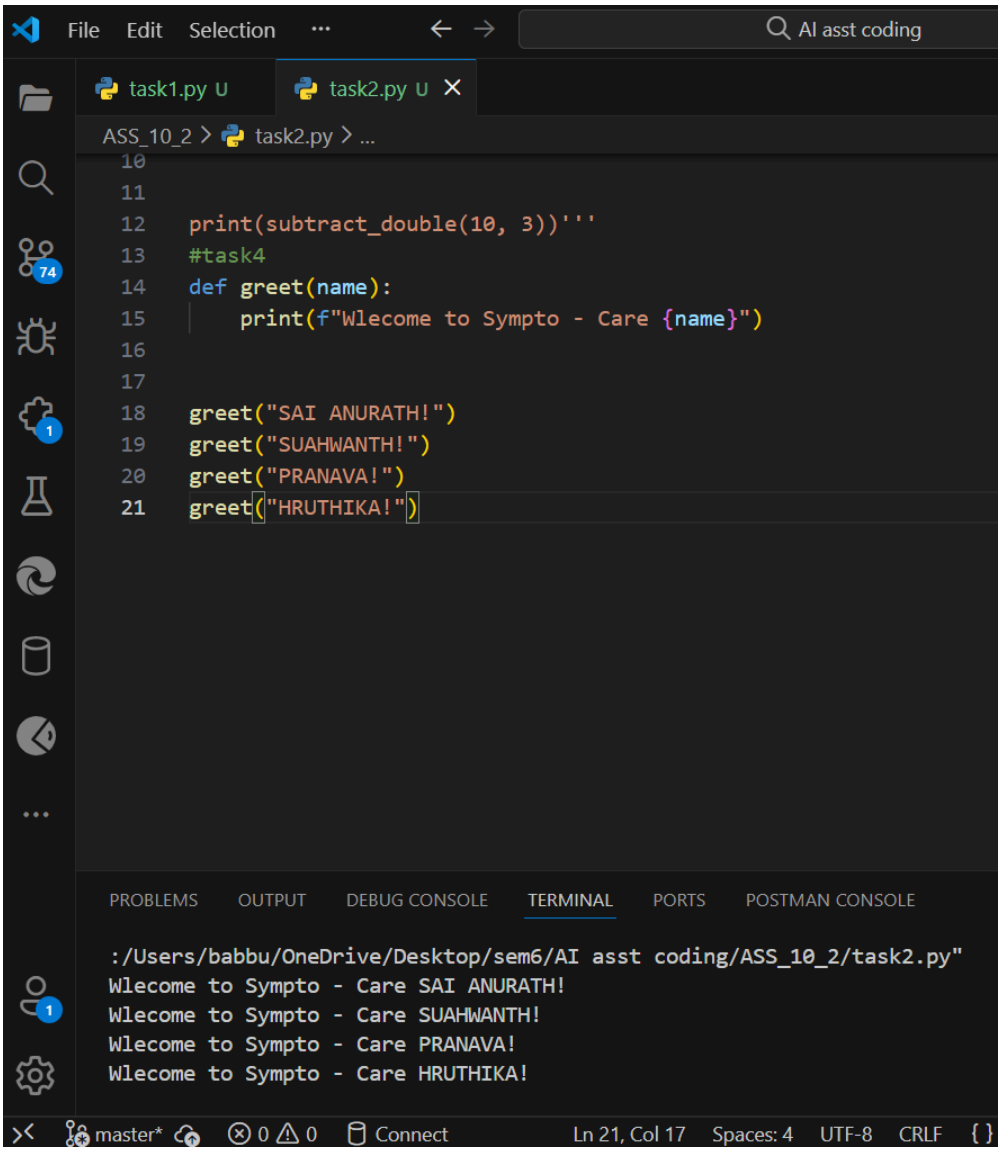
I also added proper indentation and separated the return value into a new variable called `result`, because it makes the code easier to read. The functionality is exactly the same as before; I only improved the readability and clarity.

> **Task Description -4(Structural Refactoring)**
> **Task:**
> Use AI to refactor repetitive code into reusable functions.
> **Sample Input Code:**
> print("Hello Ram")
> print("Hello Sita")
> print("Hello Ravi")
> **Expected Output-4:**
> Modular Python code using reusable functions to eliminate repetition.

```
def greet(name):
    print(f"Wlecome to Sympto - Care {name}")


greet("SAI ANURATH!")
greet("SUAHWANTH!")
```

```
greet("PRANAVA!")
greet("HRUTHIKA!")
```



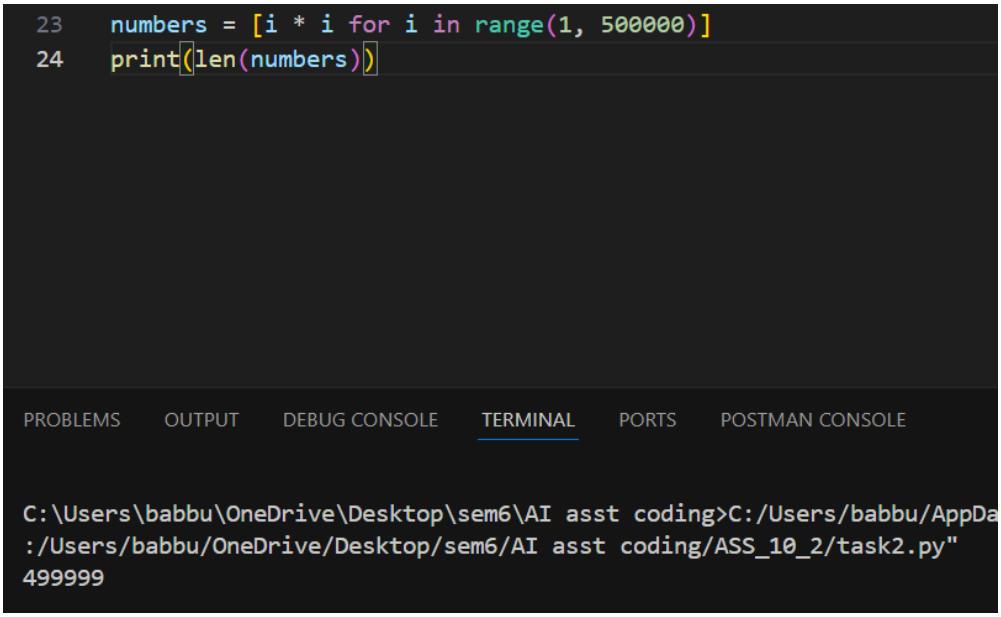
REVIEW AND EXPLAINATION

The original code was repeating the same print statement three times with only the name changing. To make it cleaner and avoid repetition, I created a function called `greet()` that takes a name as a parameter and prints the greeting. Now, instead of writing multiple print statements, I just call the function with different names. This makes the code shorter, easier to maintain, and more reusable.

> **Task Description -5(Efficiency Enhancement)**
> Task:
> Use AI to optimize Python code for better performance.
> **Sample Input Code:**
> numbers = [ ]

> for i in range(1, 500000):
> numbers.append(i * i)
> print(len(numbers))
> **Expected Output-5:**
> Optimized Python code that achieves the same result with improved performance.

```
numbers = [i * i for i in range(1, 500000)]
print(len(numbers))
```



REVIEW AND EXPLAINATION

In the original code, the list was created using a for-loop and the `append()` method. This works, but it's slower because each append happens one at a time. I replaced the loop with a list comprehension, which is faster and more efficient in Python because it handles the whole list creation internally.

The functionality stays the same: it still stores the squares of numbers from 1 to 499999 and prints the length. The only change is the method of creating the list, which improves performance and makes the code cleaner.